

Password Strength Analyzer with Custom Wordlist Generator

Introduction:

Passwords are the first line of defense in most digital systems. However, weak passwords remain one of the most common security vulnerabilities. Attackers often use dictionary attacks and custom wordlists to guess passwords efficiently. This project focuses on analyzing password strength and generating customized wordlists based on user-specific information. The tool is designed as a command-line application that helps users understand how secure their passwords are and demonstrates how attackers create targeted wordlists.

Objectives:

The main objectives of this project are:

- To analyze password strength using modern techniques
- To estimate password cracking time
- To generate custom wordlists based on user input
- To simulate real-world password attack strategies
- To provide a simple and effective CLI-based tool

Technologies Used:

- Python
- argparse (for command-line interface)
- zxcvbn (password strength estimator)
- itertools (for generating combinations)

System Design:

The system consists of two main modules:

Password Analyzer:

This module uses the zxcvbn library to evaluate password strength. It provides:

- Strength score (0 to 4)
- Estimated crack time
- Feedback for improvement

Wordlist Generator:

This module generates custom wordlists using:

- User inputs (name, date, pet, etc.)
- Leetspeak substitutions
- Word combinations
- Common number patterns

Key Features:

Password Strength Analysis:

The tool evaluates password strength and provides feedback to help users create stronger passwords.

Custom Wordlist Generation:

Generates attack-specific wordlists using personal information, mimicking real-world hacking techniques.

Leetspeak Conversion

Transforms characters into common substitutions such as:

- a → @, 4
- e → 3
- → 0

Pattern Addition:

Appends commonly used numbers such as:

- Years (2023, 2024, 2025)
- Numeric patterns (123, 1234)

File Export

Generated wordlists are saved in .txt format for use in security testing tools.

Working Principle

1. User provides a password or inputs
2. Password is analyzed using zxcvbn
3. Strength and crack time are displayed
4. Inputs are processed to generate variations
5. Wordlist is created using combinations and patterns
6. Final list is saved to a file

Results:

The tool successfully analyzes password strength and generates customized wordlists. It demonstrates how weak passwords can be easily guessed using targeted attack techniques.

Conclusion:

This project highlights the importance of strong password practices and demonstrates how attackers exploit weak passwords. It provides practical insight into password security and attack methodologies.